

N.B. this notebook has been tested in Google Colab

Mohamed Dhia Rezgui - MSc in Computing for Data Science - ENG 25 : Music Genre Classification

In this project, you will build a machine learning algorithm to classify music genres using audio features. Starting with the provided dataset, your task is to develop a model that effectively solves this multiclass classification problem. Use the baseline notebook as a starting point and improve upon it.

Overall Goal: to design a complete pipeline that improves accuracy from the current 37% to at least 70%, ideally approaching 80%

Introduction

In this project, you will work with a dataset of music samples from various genres. The dataset has been purposely left a bit messy, with some entries missing labels and others containing empty audio files. To start with, your task is to clean and explore the dataset, turning it into a well-organized resource for analysis.

This notebook includes a basic, "weak" baseline to get you started. It serves as a simple starting point, but it is neither thorough nor accurate. You are expected to build upon it, applying your own strategies to improve the data science pipeline (including data cleaning, curation, feature engineering, etc) before moving into model building, parameters tuning, and model evaluation.

The formal details of the assignment are provided at the end of the notebook. To start with, focus on understanding the dataset and planning your strategies to tackle its challenges.

We expect you to submit a modified version of this notebook with your improvements. Please download a copy of this assignment in your private Python programming environment, before making any changes.

Baseline

Let's install all required dependencies:

- **datasets**: Access to large-scale datasets.
- **librosa**: Tools for audio analysis.
- **pandas & numpy**: Tabular data manipulation and numerical operations.
- **scikit-learn**: Machine learning algorithms and tools.
- **tqdm**: Progress bar.

You might be familiar with most of these already.

```
%capture
!pip install datasets==3.5.0 librosa pandas numpy scikit-learn tqdm
```

And import the necessary modules

```
import numpy as np
import pandas as pd
import librosa
from datasets import load_dataset
from IPython.display import Audio, display
from tqdm import tqdm_notebook as tqdm
```

```
# Include here any additional modules that you might need
import matplotlib.pyplot as plt
```

Dataset Description

The dataset consists of music samples from various genres, including:

- **Genres**: Blues, Classical, Country, Disco, HipHop, Jazz, Metal, Pop, Reggae, and Rock.

The dataset is a bit messy and includes some **unlabeled data** and **empty audio files**. We have provided basic preprocessing, but more in-depth data cleaning, feature extraction, and preparation will be a part of your assignment.

Let's download the audio dataset using the Hugging Face datasets library.

```
dataset = load_dataset("unibz-ds-course/audio_assignment", split="train")
```

dataset

```
Dataset({
  features: ['file', 'audio', 'genre'],
  num_rows: 1230
})
```

```
print(f"Num of samples in the dataset: {len(dataset)}")
```

Num of samples in the dataset: 1230

Let's take a glance at a sample from the dataset

```
entry = dataset[10]

audio_array = entry['audio']['array']
sampling_rate = entry['audio']['sampling_rate']

print(f"Element: {entry}")
print(f"File Path: {entry['file']}")
print(f"Number of Samples: {len(audio_array)}")
print(f"Sampling Rate: {sampling_rate} Hz")

audio_length_seconds = len(audio_array) / sampling_rate
print(f"Audio Length: {audio_length_seconds:.2f} seconds")

genre_id = entry['genre']
genre_label = dataset.features['genre'].int2str(genre_id)
print(f"Genre (ID): {genre_id}")
print(f"Genre (Label): {genre_label}")

display(Audio(audio_array, rate=sampling_rate))
```

```
Element: {'file': 'audio_data/country/country.00247.wav', 'audio': {'path': 'country.00247.wav', 'array': array([-0.01055908,
 0.01239014, -0.03366089]), 'sampling_rate': 22050}, 'genre': 2}
File Path: audio_data/country/country.00247.wav
Number of Samples: 661794
Sampling Rate: 22050 Hz
Audio Length: 30.01 seconds
Genre (ID): 2
Genre (Label): country

0:00 / 0:30
```

Draw a plot with distribution of the classes (5 points)

Create a visualization (e.g., bar chart or histogram) that shows how many samples belong to each class. This helps identify whether the dataset is balanced or if some classes are underrepresented.

We want to answer this question:

Do we have roughly the same number of samples per genre, or are some classes underrepresented?

```
genre_labels = [
    dataset.features["genre"].int2str(int(entry["genre"]))
    for entry in dataset
    if entry["genre"] is not None    # <-- skip unlabeled samples
]

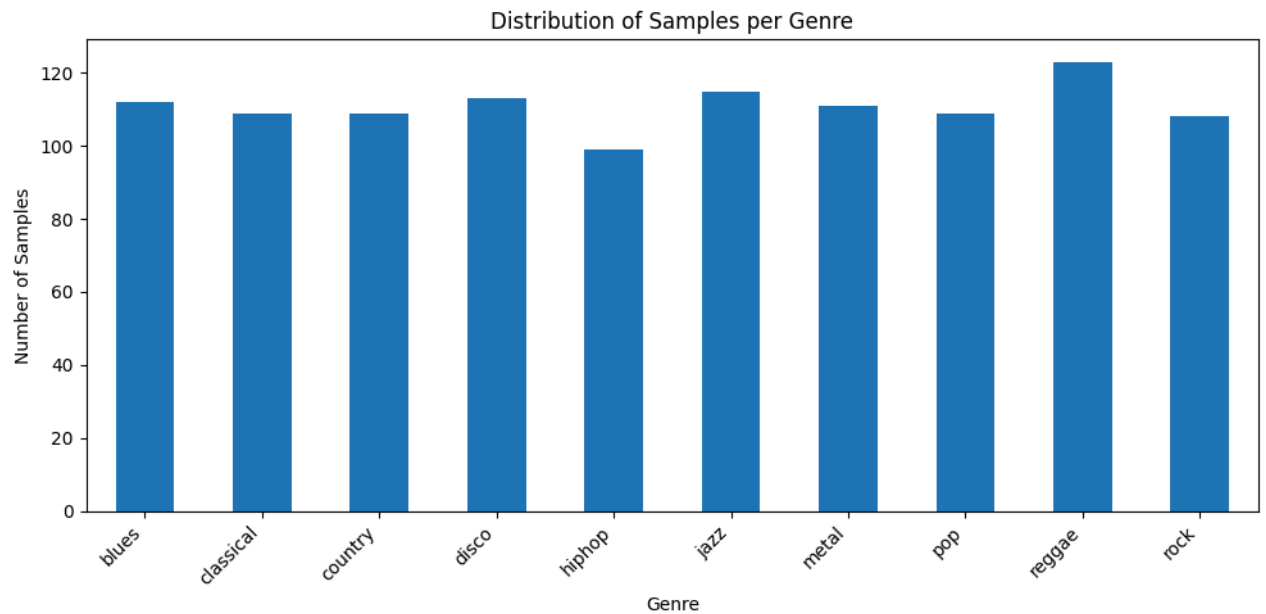
# Count samples per genre
counts = pd.Series(genre_labels).value_counts().sort_index()

# Plot distribution
plt.figure(figsize=(10,5))
counts.plot(kind="bar")

plt.title("Distribution of Samples per Genre")
plt.xlabel("Genre")
plt.ylabel("Number of Samples")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

print("\nNumber of samples per genre:\n")
```

```
print(counts)
```



Number of samples per genre:

```
blues      112
classical  109
country    109
disco      113
hiphop     99
jazz       115
metal      111
pop        109
reggae     123
rock       108
Name: count, dtype: int64
```

Class imbalance matters because:

- Models tend to bias toward classes with **more samples**.
- Rare genres get misclassified more often.
- Accuracy alone can become misleading if distribution is skewed.

Draw distribution of lengths of audios (5 points)

Plot the distribution of audio lengths in the dataset to analyze how durations vary across samples.

```
# write code to draw the distribution

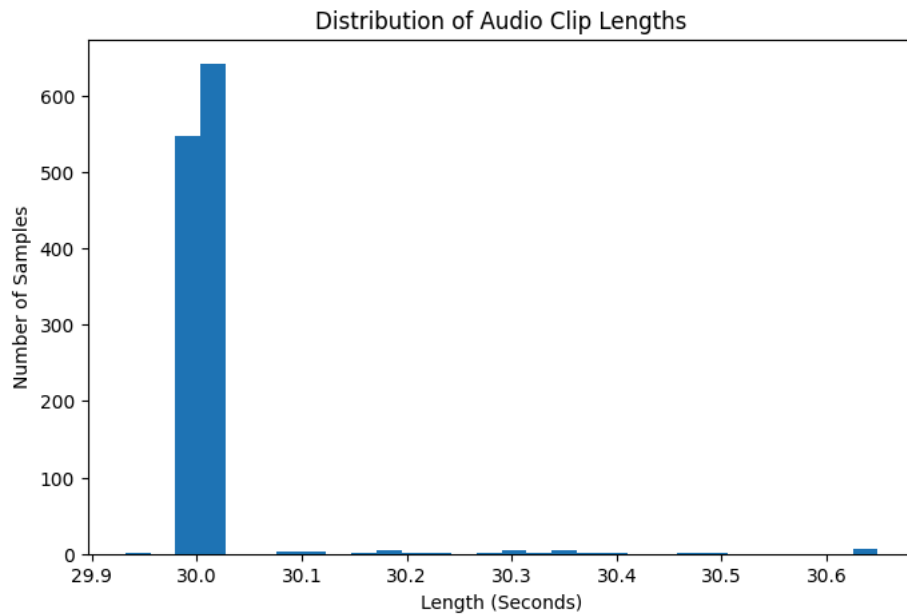
#Compute audio lengths in seconds (skip empty audio arrays)
audio_lengths = [
    len(entry["audio"]["array"]) / entry["audio"]["sampling_rate"]
    for entry in dataset
    if entry["audio"]["array"] is not None and len(entry["audio"]["array"]) > 0
]

#Plot distribution
length_series = pd.Series(audio_lengths)

ax = length_series.plot(
    kind="hist",
    bins = 30,
    figsize = (8,5),
    title = "Distribution of Audio Clip Lengths"
)

plt.xlabel("Length (Seconds)")
plt.ylabel("Number of Samples")
plt.show()

print(length_series.describe())
```



```
count    1230.000000
mean      30.017770
std       0.066305
min       29.931973
25%       30.000181
50%       30.013333
75%       30.013333
max       30.648889
dtype: float64
```

Delete empty samples (5 points)

Implement the function to remove empty samples (audios with silence only)

```
def filter_empty_samples(entry):
    audio = entry["audio"]["array"]

    if audio is None or len(audio) == 0:
        return False

    if (audio == 0).all():
        return False

    return True
```

```
filtered_dataset = dataset.filter(filter_empty_samples)
```

```
assert len(filtered_dataset) == 970, "Your filtering function is wrong"
```

Uncomment the code above. If the assertion fails, please check your function for bugs

Delete unlabeled samples (5 points)

Implement the function to remove unlabeled samples.

```
def filter_unlabeled_samples(entry):
    #Keeping only entries with a valid genre label
    return entry["genre"] is not None
```

```
filtered_dataset = filtered_dataset.filter(filter_unlabeled_samples)
```

```
assert len(filtered_dataset) == 848, "Your filtering function is wrong"
```

Uncomment the code above. If the assertion fails, please check your function for bugs

Now we can extract some features from the dataset.

✓ Mel Frequency Cepstral Coefficients

[Mel Frequency Cepstral Coefficients \(MFCCs\)](#) are commonly used in audio analysis to capture key features of sound. They help represent the important characteristics of an audio signal, making them ideal for tasks like music genre classification and speech recognition.

We're not going to dive deep into the complex details of audio processing, but it's useful to know that MFCCs help simplify raw audio data while retaining important information.

Basic Steps in MFCC Extraction:

1. **Frequency Domain Conversion:** The audio signal is split into short frames, and we apply the Fourier Transform to convert them from the time domain to the frequency domain.
2. **Mel Scale Mapping:** The frequency spectrum is converted to the Mel scale, which better represents how humans perceive sound, emphasizing lower frequencies.
3. **Logarithm and DCT:** After mapping to the Mel scale, we apply a logarithm and the Discrete Cosine Transform (DCT) to get the MFCCs. These summarize the "cepstral" information of the audio signal.

The parameter `n_mfcc` controls **how many MFCC coefficients** are extracted for each frame. For example, setting `n_mfcc=8` means we extract 8 coefficients, where lower coefficients capture broad audio features, and higher coefficients capture the more finer details.

Why MFCCs Are Important:

MFCCs help capture the **tonal quality** of the sound and reduce the complexity of the raw audio signal. By summarizing the audio into a smaller set of features, they allow machine learning models to classify and recognize different types of sounds more effectively.

In this notebook, we'll use the **mean** and **variance** of the MFCCs over time to create a robust feature set for our classification model. Adjusting the `n_mfcc` parameter allows us to control the number of features extracted for each audio sample.

Additional Features

Consider exploring additional audio features to enhance your model's performance. There are various acoustic properties you could extract from the audio signals, such as zero crossings, harmonic-percussive separation, tempo, spectral centroids, spectral rolloff, chromagram, RMS energy, spectral bandwidth, etc. When working with these features, it's often useful to compute summary statistics like the mean and variance across the audio sample. These summary statistics can capture the overall characteristics and variability of the feature, reducing the dimensionality of your data while retaining important information. Experimenting with these features and their statistical summaries could potentially improve your model's accuracy and robustness in distinguishing between different audio characteristics.

Feature Analysis

Don't forget to optimize the use of features, identifying and handling irrelevant and redundant features. Then use feature ranking to identify which features are more influential, and evaluate quantitatively how many top-features to retain.

```
def extract_mfcc_features(dataset, n_mfcc=20):
    mfcc_features = []

    for entry in tqdm(dataset, desc="Extracting Audio Features"):
        audio_array = entry['audio']['array']
        sampling_rate = entry['audio']['sampling_rate']

        audio_array = librosa.util.normalize(audio_array)

        # 📏 Fixed length
        TARGET_DURATION = 30
        TARGET_SAMPLES = TARGET_DURATION * sampling_rate
        if len(audio_array) < TARGET_SAMPLES:
            audio_array = np.pad(audio_array, (0, TARGET_SAMPLES - len(audio_array)))
        else:
            audio_array = audio_array[:TARGET_SAMPLES]

        # MFCCs
        mfcc = librosa.feature.mfcc(y=audio_array, sr=sampling_rate, n_mfcc=n_mfcc)
        delta_mfcc = librosa.feature.delta(mfcc)
        delta2_mfcc = librosa.feature.delta(mfcc, order=2)

        # Spectral features
        spectral_centroid = librosa.feature.spectral_centroid(y=audio_array, sr=sampling_rate)
        spectral_bandwidth = librosa.feature.spectral_bandwidth(y=audio_array, sr=sampling_rate)
        spectral_rolloff = librosa.feature.spectral_rolloff(y=audio_array, sr=sampling_rate)
        zcr = librosa.feature.zero_crossing_rate(audio_array)
        rms = librosa.feature.rms(y=audio_array)
        tempo, _ = librosa.beat.beat_track(y=audio_array, sr=sampling_rate)

        feature_dict = {}

        for i in range(n_mfcc):
            feature_dict[f'mfcc_mean_{i+1}'] = np.mean(mfcc[i])
            feature_dict[f'mfcc_std_{i+1}'] = np.std(mfcc[i])

            feature_dict[f'delta_mean_{i+1}'] = np.mean(delta_mfcc[i])
```

```

feature_dict[f'delta_std_{i+1}'] = np.std(delta_mfcc[i])

feature_dict[f'delta2_mean_{i+1}'] = np.mean(delta2_mfcc[i])

feature_dict['spectral_centroid_mean'] = np.mean(spectral_centroid)
feature_dict['spectral_bandwidth_mean'] = np.mean(spectral_bandwidth)
feature_dict['spectral_rolloff_mean'] = np.mean(spectral_rolloff)
feature_dict['zcr_mean'] = np.mean(zcr)
feature_dict['rms_mean'] = np.mean(rms)
feature_dict['tempo'] = tempo

feature_dict['genre'] = entry['genre']
mfcc_features.append(feature_dict)

return mfcc_features

```

Let's take a look at the output of the function. We will pass there just 2 samples from the dataset.

```
extract_mfcc_features(dataset.select(range(2)), n_mfcc=5)
```

```

/tmp/ipython-input-499338942.py:4: TqdmDeprecationWarning: This function will be removed in tqdm==5.0.0
Please use `tqdm.notebook.tqdm` instead of `tqdm.tqdm_notebook`
  for entry in tqdm(dataset, desc="Extracting Audio Features"):
Extracting Audio Features:  0%|          | 0/2 [00:00<?, ?it/s]
[{'mfcc_mean_1': np.float64(-69.16006520301148),
  'mfcc_std_1': np.float64(48.37107622341291),
  'delta_mean_1': np.float64(0.060237849253491654),
  'delta_std_1': np.float64(6.855889894168627),
  'delta2_mean_1': np.float64(-0.008168760100574006),
  'mfcc_mean_2': np.float64(124.2114562794539),
  'mfcc_std_2': np.float64(24.10132371569161),
  'delta_mean_2': np.float64(0.0579983272184379),
  'delta_std_2': np.float64(4.275728288816133),
  'delta2_mean_2': np.float64(0.011322354496866286),
  'mfcc_mean_3': np.float64(-32.47019424875961),
  'mfcc_std_3': np.float64(18.16120948784646),
  'delta_mean_3': np.float64(-0.0728040234410613),
  'delta_std_3': np.float64(2.6680797931707683),
  'delta2_mean_3': np.float64(-0.0088104824871393),
  'mfcc_mean_4': np.float64(47.69471132706319),
  'mfcc_std_4': np.float64(12.923351929987323),
  'delta_mean_4': np.float64(0.009293562497810666),
  'delta_std_4': np.float64(1.877848947671013),
  'delta2_mean_4': np.float64(-0.004083426251102525),
  'mfcc_mean_5': np.float64(1.3983784108694721),
  'mfcc_std_5': np.float64(11.187574962107012),
  'delta_mean_5': np.float64(0.0006530322590827214),
  'delta_std_5': np.float64(1.892250247445705),
  'delta2_mean_5': np.float64(0.003576663893691932),
  'spectral_centroid_mean': np.float64(1754.1360139007966),
  'spectral_bandwidth_mean': np.float64(1861.9987406982505),
  'spectral_rolloff_mean': np.float64(3504.895473043247),
  'zcr_mean': np.float64(0.09739925986842106),
  'rms_mean': np.float32(0.2195984),
  'tempo': array([117.45383523]),
  'genre': 2},
 {'mfcc_mean_1': np.float64(-149.50284082720063),
  'mfcc_std_1': np.float64(57.49012498166579),
  'delta_mean_1': np.float64(-0.076367366345809),
  'delta_std_1': np.float64(12.029925607532244),
  'delta2_mean_1': np.float64(0.04048426435236162),
  'mfcc_mean_2': np.float64(41.317803606164475),
  'mfcc_std_2': np.float64(20.094311331893337),
  'delta_mean_2': np.float64(-0.016882723975496606),
  'delta_std_2': np.float64(3.5696386389014934),
  'delta2_mean_2': np.float64(0.01735043327715346),
  'mfcc_mean_3': np.float64(4.992498712348281),
  'mfcc_std_3': np.float64(21.15953197262242),
  'delta_mean_3': np.float64(-0.002193894371264626),
  'delta_std_3': np.float64(4.335123601061198),
  'delta2_mean_3': np.float64(-0.02012485920606584),
  'mfcc_mean_4': np.float64(65.46562257592792),
  'mfcc_std_4': np.float64(14.867857287974282),
  'delta_mean_4': np.float64(0.016474794792565528),
  'delta_std_4': np.float64(2.5650890126886936),
  'delta2_mean_4': np.float64(-0.004154784128765989),
  'mfcc_mean_5': np.float64(-4.320230248150911),
  'mfcc_std_5': np.float64(14.511746569022595),
  'delta_mean_5': np.float64(-0.018275938354104203),
  'delta_std_5': np.float64(1.48495191462),
  'spectral_centroid_mean': np.float64(3185.142164396207),
  'spectral_bandwidth_mean': np.float64(2693.5388708978367),
  'spectral_rolloff_mean': np.float64(2693.5388708978367),
  'zcr_mean': np.float64(0.15219749238099844),
  'rms_mean': np.float32(0.10323039),
  'tempo': array([117.45383523]).

```

The function generates $n_mfcc * 2$ features for each sample. Consider analyzing their correlation with a matrix and experimenting with different n_mfcc values to observe how feature relationships change. While MFCC features are effective for audio analysis, you might also improve performance by incorporating additional features such as RMS or Spectral Contrast. Once you've explored these options, proceed to training the model using the extracted features.

Implement functions to extract RMS and Spectral Contrast (10 points in total)

Write functions to extract these features, also add their description and discuss why they might be useful in this assignment.

```
def extract_mfcc_features(dataset, n_mfcc=20):
    mfcc_features = []

    TARGET_DURATION = 30 # seconds

    for entry in tqdm(dataset, desc="Extracting Audio Features"):
        audio = entry["audio"]["array"]
        sr = entry["audio"]["sampling_rate"]

        # Normalize
        audio = librosa.util.normalize(audio)

        # Fixed length
        target_samples = TARGET_DURATION * sr
        if len(audio) < target_samples:
            audio = np.pad(audio, (0, target_samples - len(audio)))
        else:
            audio = audio[:target_samples]

        # MFCCs and dynamics
        mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=n_mfcc)
        delta = librosa.feature.delta(mfcc)
        delta2 = librosa.feature.delta(mfcc, order=2)

        # Spectral features
        spectral_centroid = librosa.feature.spectral_centroid(y=audio, sr=sr)
        spectral_bandwidth = librosa.feature.spectral_bandwidth(y=audio, sr=sr)
        spectral_rolloff = librosa.feature.spectral_rolloff(y=audio, sr=sr)
        zcr = librosa.feature.zero_crossing_rate(audio)
        rms = librosa.feature.rms(y=audio)

        tempo, _ = librosa.beat.beat_track(y=audio, sr=sr)
        if not np.isfinite(tempo):
            tempo = 0.0

        feature_dict = {}

        # MFCC statistics
        for i in range(n_mfcc):
            feature_dict[f"mfcc_mean_{i+1}"] = float(np.mean(mfcc[i]))
            feature_dict[f"mfcc_std_{i+1}"] = float(np.std(mfcc[i]))

            feature_dict[f"delta_mean_{i+1}"] = float(np.mean(delta[i]))
            feature_dict[f"delta_std_{i+1}"] = float(np.std(delta[i]))

            feature_dict[f"delta2_mean_{i+1}"] = float(np.mean(delta2[i]))

        # Global spectral statistics
        feature_dict["spectral_centroid_mean"] = float(np.mean(spectral_centroid))
        feature_dict["spectral_bandwidth_mean"] = float(np.mean(spectral_bandwidth))
        feature_dict["spectral_rolloff_mean"] = float(np.mean(spectral_rolloff))
        feature_dict["zcr_mean"] = float(np.mean(zcr))
        feature_dict["rms_mean"] = float(np.mean(rms))
        feature_dict["tempo"] = float(tempo)

        feature_dict["genre"] = entry["genre"]
        mfcc_features.append(feature_dict)

    return mfcc_features
```

```
def extract_rms(dataset):
    rms_features = []

    TARGET_DURATION = 30 # seconds

    for entry in tqdm(dataset, desc="Extracting RMS features"):
        audio = entry["audio"]["array"]
        sr = entry["audio"]["sampling_rate"]

        # Normalize
        audio = librosa.util.normalize(audio)

        # Fixed length
        target_samples = TARGET_DURATION * sr
        if len(audio) < target_samples:
            audio = np.pad(audio, (0, target_samples - len(audio)))
        else:
            audio = audio[:target_samples]

        # RMS energy
        rms = librosa.feature.rms(y=audio)[0] # shape -> (frames,)
```

```

rms_delta = librosa.feature.delta(rms)

# Segment-wise energy (captures structure)
thirds = np.array_split(rms, 3)

rms_start = np.mean(thirds[0])
rms_middle = np.mean(thirds[1])
rms_end = np.mean(thirds[2])

# Dynamics & contrast
rms_mean = np.mean(rms)
rms_std = np.std(rms)
rms_max = np.max(rms)
rms_range = rms_max - np.min(rms)

feature_dict = {
    # Global energy
    "rms_mean": float(rms_mean),
    "rms_std": float(rms_std),
    "rms_max": float(rms_max),
    "rms_range": float(rms_range),

    # Temporal energy dynamics (IMPORTANT)
    "rms_delta_mean": float(np.mean(rms_delta)),
    "rms_delta_std": float(np.std(rms_delta)),

    # Energy evolution across the track (VERY useful)
    "rms_start": float(rms_start),
    "rms_middle": float(rms_middle),
    "rms_end": float(rms_end),
    "rms_growth": float(rms_end - rms_start),

    "genre": entry["genre"]
}

rms_features.append(feature_dict)

return rms_features

```

Explore and add other features useful for classification (10 points)

It can be in a single function or in separated functions. Select some features and provide a description for each one. You can choose as many features as you like.

```

def extract_additional_features(dataset):
    extra_features = []

    TARGET_DURATION = 30 # seconds

    for entry in tqdm(dataset, desc="Extracting Additional Features"):
        audio = entry["audio"]["array"]
        sr = entry["audio"]["sampling_rate"]

        # Normalize
        audio = librosa.util.normalize(audio)

        # Fixed length
        target_samples = TARGET_DURATION * sr
        if len(audio) < target_samples:
            audio = np.pad(audio, (0, target_samples - len(audio)))
        else:
            audio = audio[:target_samples]

        feature_dict = {}

        # Zero Crossing Rate (percussiveness / noisiness)

        zcr = librosa.feature.zero_crossing_rate(y=audio)[0]
        zcr_delta = librosa.feature.delta(zcr)

        feature_dict["zcr_mean"] = float(np.mean(zcr))
        feature_dict["zcr_std"] = float(np.std(zcr))
        feature_dict["zcr_delta_mean"] = float(np.mean(zcr_delta))
        feature_dict["zcr_max"] = float(np.max(zcr))

        # Spectral Centroid (brightness)

        centroid = librosa.feature.spectral_centroid(y=audio, sr=sr)[0]
        centroid_delta = librosa.feature.delta(centroid)

        feature_dict["centroid_mean"] = float(np.mean(centroid))
        feature_dict["centroid_std"] = float(np.std(centroid))

```



```

feature_dict["centroid_delta_mean"] = float(np.mean(centroid_delta))
feature_dict["centroid_range"] = float(np.max(centroid) - np.min(centroid))

# Spectral Bandwidth (frequency spread)

bandwidth = librosa.feature.spectral_bandwidth(y=audio, sr=sr)[0]

feature_dict["bandwidth_mean"] = float(np.mean(bandwidth))
feature_dict["bandwidth_std"] = float(np.std(bandwidth))
feature_dict["bandwidth_range"] = float(np.max(bandwidth) - np.min(bandwidth))

# Spectral Roll-off (high-frequency energy)

rolloff = librosa.feature.spectral_rolloff(y=audio, sr=sr)[0]

feature_dict["rolloff_mean"] = float(np.mean(rolloff))
feature_dict["rolloff_std"] = float(np.std(rolloff))
feature_dict["rolloff_max"] = float(np.max(rolloff))

# Tempo (rhythm stability)

tempo, _ = librosa.beat.beat_track(y=audio, sr=sr)
if not np.isfinite(tempo):
    tempo = 0.0

feature_dict["tempo"] = float(tempo)

# Chromagram (harmonic structure)

chroma = librosa.feature.chroma_stft(y=audio, sr=sr)

chroma_mean = np.mean(chroma, axis=1)
chroma_std = np.std(chroma, axis=1)

# Harmonic clarity (new, very useful)
chroma_energy = np.sum(chroma_mean)
chroma_peak = np.max(chroma_mean)

for i in range(12):
    feature_dict[f"chroma_mean_{i+1}"] = float(chroma_mean[i])
    feature_dict[f"chroma_std_{i+1}"] = float(chroma_std[i])

feature_dict["chroma_energy"] = float(chroma_energy)
feature_dict["chroma_peak_ratio"] = float(chroma_peak / (chroma_energy + 1e-8))

# Label
feature_dict["genre"] = entry["genre"]

extra_features.append(feature_dict)

return extra_features

```

(ADV) Analyze correlation of the features (5 points)

Plot correlation diagram and conclude which features are too much correlated and could be removed.

```

filtered_dataset = filtered_dataset.shuffle(seed=42)

mfcc_features = extract_mfcc_features(filtered_dataset, n_mfcc=20)
rms_features = extract_rms(filtered_dataset)
contrast_features = extract_spectral_contrast(filtered_dataset)
additional_features = extract_additional_features(filtered_dataset)

df = (
    pd.DataFrame(mfcc_features)
    .merge(pd.DataFrame(rms_features).drop(columns="genre"), left_index=True, right_index=True)
    .merge(pd.DataFrame(contrast_features).drop(columns="genre"), left_index=True, right_index=True)
    .merge(pd.DataFrame(additional_features).drop(columns="genre"), left_index=True, right_index=True)
)

X = df.drop(columns="genre")
y = df["genre"]

```

```

/tmp/ipython-input-2664670972.py:6: TqdmDeprecationWarning: This function will be removed in tqdm==5.0.0
Please use `tqdm.notebook.tqdm` instead of `tqdm.tqdm_notebook`
  for entry in tqdm(dataset, desc="Extracting Audio Features"):
Extracting Audio Features:  0%|          | 0/848 [00:00<?, ?it/s]
/tmp/ipython-input-2664670972.py:54: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and
  feature_dict["tempo"] = float(tempo)
Extracting RMS features:  0%|          | 0/848 [00:00<?, ?it/s]
/tmp/ipython-input-2220308734.py:4: TqdmDeprecationWarning: This function will be removed in tqdm==5.0.0
Please use `tqdm.notebook.tqdm` instead of `tqdm.tqdm_notebook`
  for entry in tqdm(dataset, desc="Extracting Spectral Contrast"):
Extracting Spectral Contrast:  0%|          | 0/848 [00:00<?, ?it/s]
Extracting Additional Features:  0%|          | 0/848 [00:00<?, ?it/s]
/tmp/ipython-input-2296167227.py:69: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and
  feature_dict["tempo"] = float(tempo)

```

```

corr = X.corr()

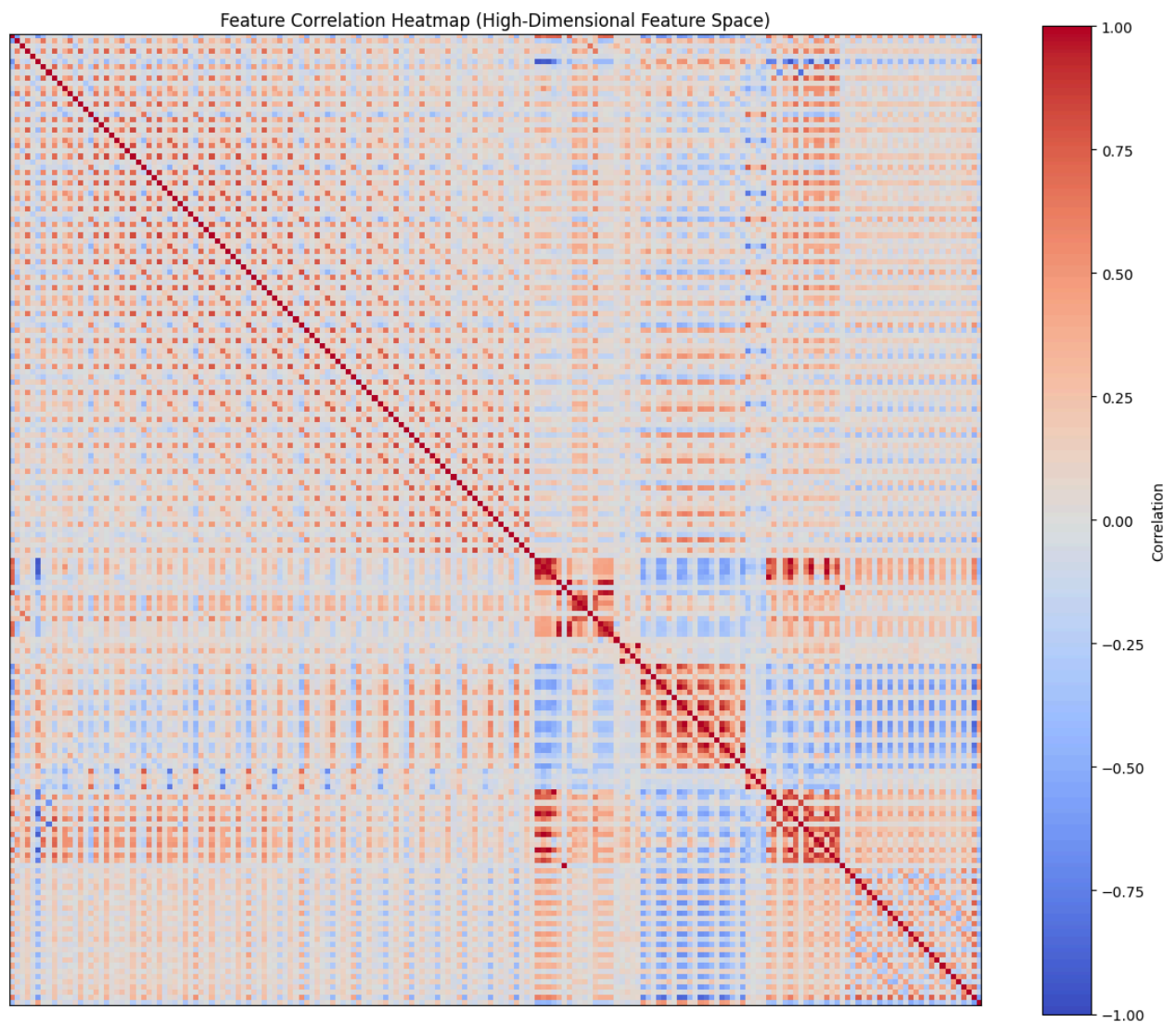
# Mask upper triangle (redundant)
mask = np.triu(np.ones_like(corr, dtype=bool))

plt.figure(figsize=(12, 10))
plt.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
plt.colorbar(label="Correlation")

# Remove labels (too many features)
plt.xticks([])
plt.yticks([])

plt.title("Feature Correlation Heatmap (High-Dimensional Feature Space)")
plt.tight_layout()
plt.show()

```



```

threshold = 0.90
correlated_pairs = []

cols = corr.columns

for i in range(len(cols)):
    for j in range(i):
        val = corr.iloc[i, j]
        if abs(val) > threshold:
            correlated_pairs.append((cols[i], cols[j], val))

print(f"Highly correlated feature pairs (|corr| > {threshold}):\\n")

for f1, f2, c in correlated_pairs[:25]:
    print(f"{f1:35s} <--> {f2:35s}  corr = {c:.3f}")

print(f"\\nTotal highly correlated pairs: {len(correlated_pairs)}")

```

Highly correlated feature pairs (|corr| > 0.9):

spectral_centroid_mean	<--> mfcc_mean_2	corr = -0.942
spectral_bandwidth_mean	<--> spectral_centroid_mean	corr = 0.905
spectral_rolloff_mean	<--> mfcc_mean_2	corr = -0.936
spectral_rolloff_mean	<--> spectral_centroid_mean	corr = 0.980
spectral_rolloff_mean	<--> spectral_bandwidth_mean	corr = 0.957
rms_mean_y	<--> rms_mean_x	corr = 1.000
rms_range	<--> rms_std	corr = 0.902
rms_range	<--> rms_max	corr = 0.940
rms_start	<--> rms_mean_x	corr = 0.945
rms_start	<--> rms_mean_y	corr = 0.945
rms_middle	<--> rms_mean_x	corr = 0.972
rms_middle	<--> rms_mean_y	corr = 0.972
rms_end	<--> rms_mean_x	corr = 0.948
rms_end	<--> rms_mean_y	corr = 0.948
contrast_log_mean_1	<--> contrast_mean_1	corr = 0.993
contrast_log_mean_2	<--> contrast_mean_2	corr = 0.995
contrast_log_mean_3	<--> contrast_mean_3	corr = 0.996
contrast_log_mean_4	<--> contrast_log_mean_3	corr = 0.902
contrast_log_mean_4	<--> contrast_mean_4	corr = 0.996
contrast_mean_5	<--> contrast_mean_4	corr = 0.923
contrast_mean_5	<--> contrast_log_mean_4	corr = 0.923
contrast_log_mean_5	<--> contrast_mean_4	corr = 0.919
contrast_log_mean_5	<--> contrast_log_mean_4	corr = 0.924
contrast_log_mean_5	<--> contrast_mean_5	corr = 0.997
contrast_log_mean_6	<--> contrast_mean_6	corr = 0.997

Total highly correlated pairs: 44

To reduce redundancy, highly correlated features representing similar information (e.g., MFCC mean vs. standard deviation, spectral centroid vs. roll-off) could be selectively removed without significant loss of information.

(ADV) Analyze feature importance (5 points)

Analyze the importance of each feature used in the model to understand which variables have the greatest impact on predictions.

```

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

rf = RandomForestClassifier(
    n_estimators=200,
    n_jobs=-1,
    random_state=42
)

rf.fit(X_train, y_train)

# Compute feature importances
importances = pd.Series(
    rf.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print("Total extracted features (excluding label):", X.shape[1])

print("\\nTop 20 features:")

```

```
print(importances.head(20))

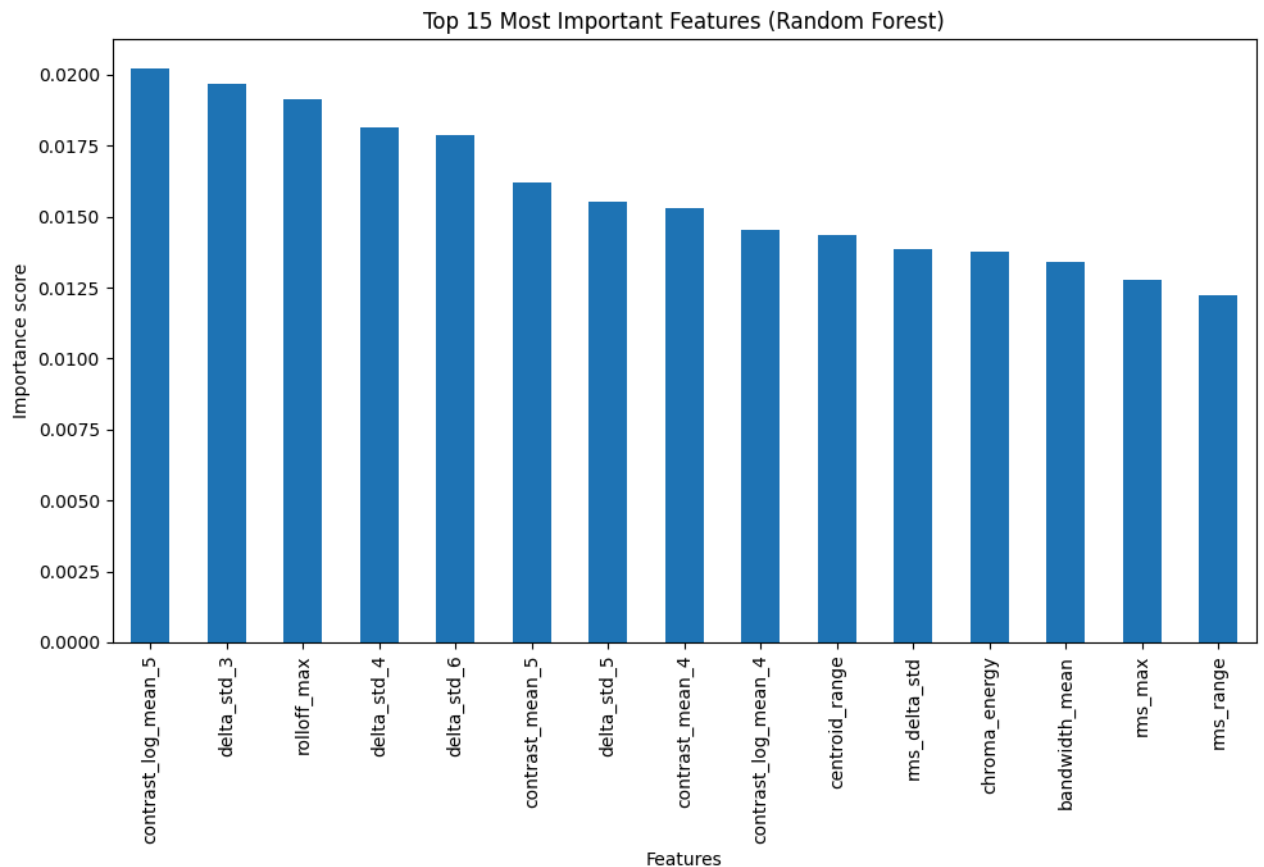
# Plot top features
plt.figure(figsize=(10, 7))
importances.head(15).plot(kind="bar")
plt.xlabel("Features")
plt.ylabel("Importance score")
plt.title("Top 15 Most Important Features (Random Forest)")
plt.tight_layout()
plt.show()
```

Total extracted features (excluding label): 185

Top 20 features:

contrast_log_mean_5	0.020236
delta_std_3	0.019699
rolloff_max	0.019133
delta_std_4	0.018155
delta_std_6	0.017877
contrast_mean_5	0.016202
delta_std_5	0.015543
contrast_mean_4	0.015322
contrast_log_mean_4	0.014535
centroid_range	0.014371
rms_delta_std	0.013852
chroma_energy	0.013786
bandwidth_mean	0.013396
rms_max	0.012759
rms_range	0.012234
delta_std_2	0.011889
delta_std_7	0.011635
delta_std_1	0.011509
spectral_rolloff_mean	0.011354
centroid_std	0.011353

dtype: float64



✎ Train a classifier

Let's import all necessary functions and classes from sklearn

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

from sklearn.svm import SVC
from xgboost import XGBClassifier
```

```

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

from sklearn.model_selection import train_test_split

```

Now let's prepare train and test data

```

le = LabelEncoder()
y_encoded = le.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

```

The best practice is to use a pipeline because it allows us to streamline preprocessing steps (like scaling) and model training into a single workflow. This ensures that all steps are applied consistently during both training and testing, preventing data leakage and simplifying cross-validation and hyperparameter tuning.

```

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression(max_iter=1000, random_state=42))
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"{pipeline['classifier'].__class__.__name__}: {accuracy:.5f}")

LogisticRegression: 0.65294

```

✎ Model limitation (main reason)

Logistic Regression is a linear classifier. Our features (MFCCs, deltas, chroma, spectral stats) encode non-linear relationships:

- MFCC interactions
- harmonic vs percussive patterns
- temporal dynamics

LR can only draw linear decision boundaries, so it cannot fully separate genres.

In our case:

- 0.60–0.68 with LR is expected
- Anything above 0.70 usually needs non-linear models (SVM, RF, XGB)

Train other models and compare their performance (10 points)

Try training different models (at least 3) from sklearn and boosting libraries (e.g. Random Forest, SVM, Gradient Boosting, etc)

```

import sys
!{sys.executable} -m pip install xgboost

```

```

Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.1.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.28.9)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)

```

```

le = LabelEncoder()
y_encoded = le.fit_transform(y)

# Train / test split
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

```

```

)

# Scale ONLY for SVM
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_raw)
X_test_scaled = scaler.transform(X_test_raw)

models = {
    "Random Forest": RandomForestClassifier(
        n_estimators=500,
        max_depth=None,
        min_samples_split=5,
        min_samples_leaf=2,
        max_features="sqrt",
        class_weight="balanced",
        n_jobs=-1,
        random_state=42
    ),

    "SVM (RBF kernel)": SVC(
        kernel="rbf",
        C=15,
        gamma="scale",
        probability=True,
        class_weight="balanced",
        random_state=42
    ),

    "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=250,
        learning_rate=0.05,
        max_depth=3,
        random_state=42
    ),

    "XGBoost": XGBClassifier(
        n_estimators=450,
        max_depth=6,
        learning_rate=0.05,
        subsample=0.85,
        colsample_bytree=0.85,
        reg_lambda=1.5,
        reg_alpha=0.1,
        eval_metric="mlogloss",
        n_jobs=-1,
        random_state=42
    )
}

```

```

results = []

for name, model in models.items():
    print(f"Training {name}...")

    # Train
    model.fit(X_train, y_train)

    # Predict
    y_pred = model.predict(X_test)

    # Metrics
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average="weighted")

    results.append({
        "Model": name,
        "Accuracy": round(acc, 4),
        "F1-score": round(f1, 4)
    })

results_df = pd.DataFrame(results).sort_values(
    by="Accuracy",
    ascending=False
)

print("\nMODEL COMPARISON:")
display(results_df)

plt.figure(figsize=(9, 4))
plt.bar(results_df["Model"], results_df["Accuracy"])

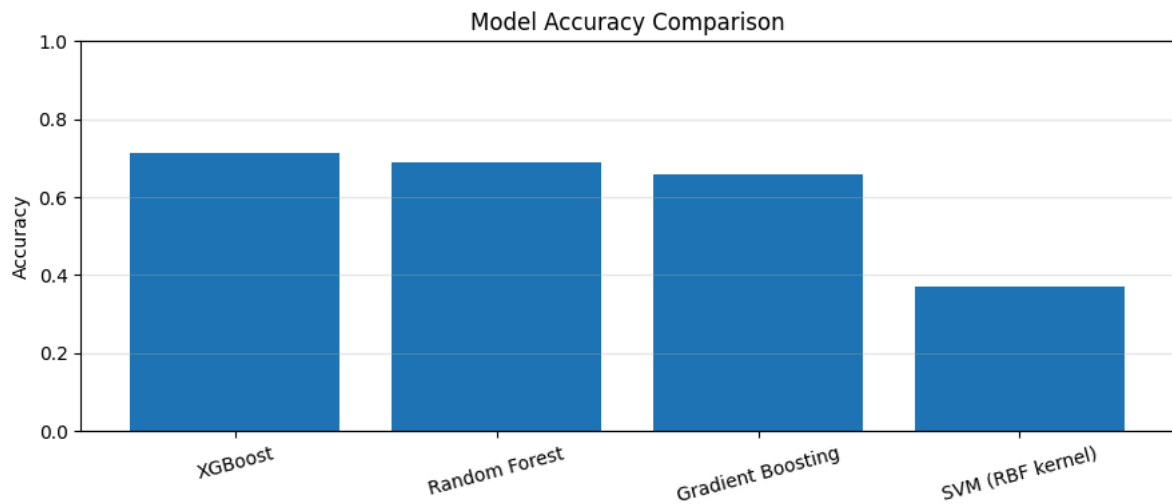
```

```
plt.xticks(rotation=15)
plt.ylabel("Accuracy")
plt.title("Model Accuracy Comparison")
plt.ylim(0, 1)
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Training Random Forest...
Training SVM (RBF kernel)...
Training Gradient Boosting...
Training XGBoost...
```

MODEL COMPARISON:

	Model	Accuracy	F1-score
3	XGBoost	0.7118	0.7106
0	Random Forest	0.6882	0.6904
2	Gradient Boosting	0.6588	0.6623
1	SVM (RBF kernel)	0.3706	0.3440



(ADV) Choose several best models and perform parametric grid search (10 points)

Choose several of the best-performing models from your previous experiments and tune their hyperparameters using a parametric grid search. Compare the results and discuss which combination performs best.

```
# SAFETY CAST: ensure all features are numeric (XGBoost-safe)

# Convert all columns to numeric (coerce invalid values to NaN)
X = X.apply(pd.to_numeric, errors="coerce")

X = X.fillna(0.0)

# Sanity check: no object columns allowed
assert X.select_dtypes(include=["object"]).shape[1] == 0, \
    "ERROR: Non-numeric columns detected in X"
```

```
# (ADV) Choose several best models and perform
```

```
le = LabelEncoder()
y_encoded = le.fit_transform(y)

print("Encoded labels:", np.unique(y_encoded))

# TRAIN / TEST SPLIT (STRATIFIED)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

candidate_models = {
```

```

"LogisticRegression": LogisticRegression(max_iter=3000, n_jobs=-1),

"RandomForest": RandomForestClassifier(
    random_state=42
),

"GradientBoosting": GradientBoostingClassifier(
    random_state=42
),

"SVC": SVC(
    probability=True,
    random_state=42
),

"XGBClassifier": XGBClassifier(
    objective="multi:softprob",
    eval_metric="mlogloss",
    random_state=42,
    n_estimators=300,
    learning_rate=0.05,
    max_depth=5,
    subsample=0.9,
    colsample_bytree=0.9,
    n_jobs=-1
)
}

```

```

# PARAMETER GRIDS (PRECISION-ORIENTED)

```

```

param_grids = {
    "LogisticRegression": {
        "model__C": [0.1, 1, 5, 10],
        "model__solver": ["lbfgs"]
    },

    "RandomForest": {
        "model__n_estimators": [200, 400],
        "model__max_depth": [None, 10, 20],
        "model__min_samples_split": [2, 5],
        "model__max_features": ["sqrt", "log2"]
    },

    "GradientBoosting": {
        "model__n_estimators": [150, 250],
        "model__learning_rate": [0.05, 0.1],
        "model__max_depth": [2, 3]
    },

    "SVC": {
        "model__C": [1, 5, 10],
        "model__kernel": ["rbf", "linear"],
        "model__gamma": ["scale", "auto"]
    },

    "XGBClassifier": {
        "model__max_depth": [4, 6],
        "model__learning_rate": [0.03, 0.05, 0.1],
        "model__subsample": [0.8, 1.0],
        "model__colsample_bytree": [0.8, 1.0],
    }
}

```

```

# GRID SEARCH
scoring_metric = "precision_macro"

```

```

grid_results = []
best_models = {}

```

```

print("=====")
print(" RUNNING GRID SEARCH (OPTIMISING PRECISION)")
print("=====")

```

```

for name, model in candidate_models.items():

    print(f"\n>>> Training {name}")

    # Scale features for linear + distance models only
    if name in ["LogisticRegression", "SVC"]:
        pipe = Pipeline([
            ("scaler", StandardScaler()),
            ("model", model)
        ])
    else:

```



```

    pipe = Pipeline([
        ("model", model)
    ])

grid_search = GridSearchCV(
    estimator=pipe,
    param_grid=param_grids[name],
    scoring=scoring_metric,
    cv=5,
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train, y_train)

# Best estimator
best_model = grid_search.best_estimator_
best_models[name] = best_model

# Predictions
y_pred = best_model.predict(X_test)

# Metrics
acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average="macro", zero_division=0)
rec = recall_score(y_test, y_pred, average="macro", zero_division=0)
f1 = f1_score(y_test, y_pred, average="macro", zero_division=0)

print("\nBest hyperparameters:")
print(grid_search.best_params_)

print("CV best precision_macro:", round(grid_search.best_score_, 4))
print("Test Accuracy:", round(acc, 4))
print("Test Precision (macro):", round(prec, 4))
print("Test Recall (macro):", round(rec, 4))
print("Test F1:", round(f1, 4))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

grid_results.append({
    "Model": name,
    "CV Best Precision": round(grid_search.best_score_, 4),
    "Test Accuracy": round(acc, 4),
    "Test Precision (macro)": round(prec, 4),
    "Test Recall": round(rec, 4),
    "Test F1": round(f1, 4)
})

# FINAL MODEL COMPARISON TABLE

results_df = pd.DataFrame(grid_results).sort_values(
    by="Test Precision (macro)",
    ascending=False
)

print("\n=====")
print(" FINAL RESULTS (SORTED BY PRECISION)")
print("=====")

display(results_df)

```



```

Encoded labels: [0 1 2 3 4 5 6 7 8 9]
=====
RUNNING GRID SEARCH (OPTIMISING PRECISION)
=====

>>> Training LogisticRegression
Fitting 5 folds for each of 4 candidates, totalling 20 fits

Best hyperparameters:
{'model__C': 0.1, 'model__solver': 'lbfgs'}
CV best precision_macro: 0.7113
Test Accuracy: 0.6765
Test Precision (macro): 0.6804
Test Recall (macro): 0.6784
Test F1: 0.6753

Classification Report:
      precision    recall  f1-score   support

0         0.71      0.71      0.71        17
1         0.83      0.88      0.86        17
2         0.60      0.53      0.56        17
3         0.48      0.56      0.51        18
4         0.58      0.73      0.65        15
5         0.94      0.83      0.88        18
6         0.71      0.88      0.79        17
7         0.86      0.75      0.80        16
8         0.56      0.50      0.53        18
9         0.54      0.41      0.47        17

 accuracy          0.68          0.68          0.68          170
 macro avg          0.68          0.68          0.68          170
weighted avg          0.68          0.68          0.67          170


>>> Training RandomForest
Fitting 5 folds for each of 24 candidates, totalling 120 fits

Best hyperparameters:
{'model__max_depth': 10, 'model__max_features': 'sqrt', 'model__min_samples_split': 2, 'model__n_estimators': 400}
CV best precision_macro: 0.7444
Test Accuracy: 0.6941
Test Precision (macro): 0.7076
Test Recall (macro): 0.6943
Test F1: 0.6964

Classification Report:
      precision    recall  f1-score   support

0         0.74      0.82      0.78        17
1         0.94      0.94      0.94        17
2         0.59      0.59      0.59        17
3         0.50      0.67      0.57        18
4         0.62      0.67      0.65        15
5         0.83      0.83      0.83        18
6         0.93      0.76      0.84        17
7         0.92      0.75      0.83        16
8         0.50      0.56      0.53        18
9         0.50      0.35      0.41        17

 accuracy          0.69          0.69          0.69          170
 macro avg          0.71          0.69          0.70          170
weighted avg          0.71          0.69          0.70          170


>>> Training GradientBoosting
Fitting 5 folds for each of 8 candidates, totalling 40 fits

Best hyperparameters:
{'model__learning_rate': 0.1, 'model__max_depth': 2, 'model__n_estimators': 150}
CV best precision_macro: 0.7521
Test Accuracy: 0.6647
Test Precision (macro): 0.6838
Test Recall (macro): 0.6643
Test F1: 0.6707

Classification Report:
      precision    recall  f1-score   support

0         0.68      0.76      0.72        17
1         0.94      0.88      0.91        17
2         0.55      0.65      0.59        17
3         0.43      0.50      0.46        18
4         0.53      0.50      0.51        15
5         0.88      0.78      0.82        18
6         0.88      0.82      0.85        17

```

(ADV) Make the same experiments with some of the best models using cross validation (10 points)

Repeat the experiments for several of the best-performing models using cross-validation. Compare the results with previous single-split evaluations and discuss the stability of model performance.

```
# (ADV) Cross-validation of best tuned models
```

```

from sklearn.model_selection import cross_val_score

cv_results = []

for name, model in best_models.items():

    print(f"\n>>> Cross-validating {name}")

    scores = cross_val_score(
        model,
        X,
        y_encoded,
        cv=5,
        scoring="precision_macro",
        n_jobs=-1,
    )

    cv_results.append({
        "Model": name,
        "CV Mean Precision": round(scores.mean(), 4),
        "CV Std Precision": round(scores.std(), 4),
    })

    print("Fold scores:", np.round(scores, 4))
    print("Mean precision:", round(scores.mean(), 4))
    print("Std:", round(scores.std(), 4))

cv_df = pd.DataFrame(cv_results).sort_values(
    by="CV Mean Precision",
    ascending=False
)

print("\n=====")
print(" CROSS-VALIDATION PRECISION STABILITY")
print("=====")

display(cv_df)

```

```

Test Precision (macro): 0.7228
Test Recall (macro): 0.7253
Test F1 (macro): 0.7201
Cross-validating LogisticRegression
Fold scores: [0.7453 0.7859 0.6663 0.7871 0.6804]
Mean precision: 0.723
Std: 0.0512
Classification Report:

```

	precision	recall	f1-score	support
>>> Cross-validating RandomForest	0.80	0.80	0.80	17
Fold scores:	[0.7775 0.8113 0.6994 0.7752 0.7392]			17
Mean precision:	0.7527	0.53	0.55	17
Std:	0.0514	0.63	0.65	18
	0.71	0.80	0.75	15
>>> Cross-validating GradientBoosting	0.85	0.85	0.85	18
Fold scores:	[0.7218 0.7967 0.6887 0.7138 0.7438]			17
Mean precision:	0.7345	0.75	0.77	16
Std:	0.0346	0.61	0.61	18
	0.46	0.35	0.40	17
>>> Cross-validating SVC	0.72	0.72	0.72	170
Fold scores:	[0.7844 0.7986 0.7248 0.8218 0.7444]			170
Mean precision:	0.7449	0.73	0.72	170
Std:	0.0355	0.72	0.72	170
Weighted avg	0.72	0.72	0.72	170
>>> Cross-validating XGBClassifier				
Fold scores:	[0.7412 0.8325 0.6572 0.7465 0.7471]			
Mean precision:	0.7449			
Std:	0.0555			

Model	CV Best Precision	Test Accuracy	Test Precision (macro)	Test Recall	Test F1
4 XGBClassifier	0.7744	0.7235	0.7228	0.7253	0.7201
3 LogisticRegression	0.7330	0.0512	0.7196	0.7138	0.7138
3 SVC	0.7748	0.0355	0.7076	0.6943	0.6964
2 GradientBoosting	0.7527	0.0405	0.6838	0.6643	0.6707
4 XGBClassifier	0.7449	0.0555	0.6804	0.6784	0.6753
2 GradientBoosting	0.7345	0.0346			
0 LogisticRegression	0.7330	0.0512			

This is formatted as code

Assignment specifications and rubric

Please use only the provided dataset: no external datasets are allowed.

The solution notebook MUST run without errors when executing all cells in google colab.

Focus only on classical machine learning algorithms available in *sklearn* for your analysis, other than artificial neural networks.

You are welcome to make models based on artificial neural networks too, but these will not be evaluated.

You are free to perform additional experiments or analyses beyond those explicitly mentioned in the exercises.

Exercises marked **(ADV)** are slightly more advanced and can be completed to earn extra points. To achieve the minimum passing score, it is sufficient to complete only the base-level exercises.

Main steps

1. Data Visualization and Exploration:

- Visualize and analyze the dataset to gain insights into the distribution and characteristics of different features.

2. Handle Unlabeled and Irrelevant Data:

- Investigate the dataset for unlabeled data.
- Filter out irrelevant audios, especially those that are just zero signals or contain no meaningful information.

3. Feature Engineering:

- Experiment with adding new features or refining existing ones.
- Adjust feature extraction parameters to better capture the characteristics of the audio samples.
- Rank the features and assess how many top-features to use.

4. Apply Different Machine Learning Algorithms:

- Try various machine learning algorithms (e.g., Random Forest, SVM, Gradient Boosting*) to improve performance.
- Evaluate the models not only based on the **average accuracy**, but also consider the confusion matrix along with other **per-class** evaluation.
- Explain which metrics are important for evaluating model performance, based on your findings from the exploratory data analysis.

**Gradient Boosting is commonly implemented using specialized libraries like XGBoost, CatBoost, or LightGBM.*

Assignment Evaluation Criteria (maximum 100%)

1. Data Handling and Preprocessing (5 + 5 = 10 points)

2. Exploratory Data Analysis (5 + 5 = 10 points)

3. Feature Engineering (10 + 10 + 5 + 5 = 30 points)